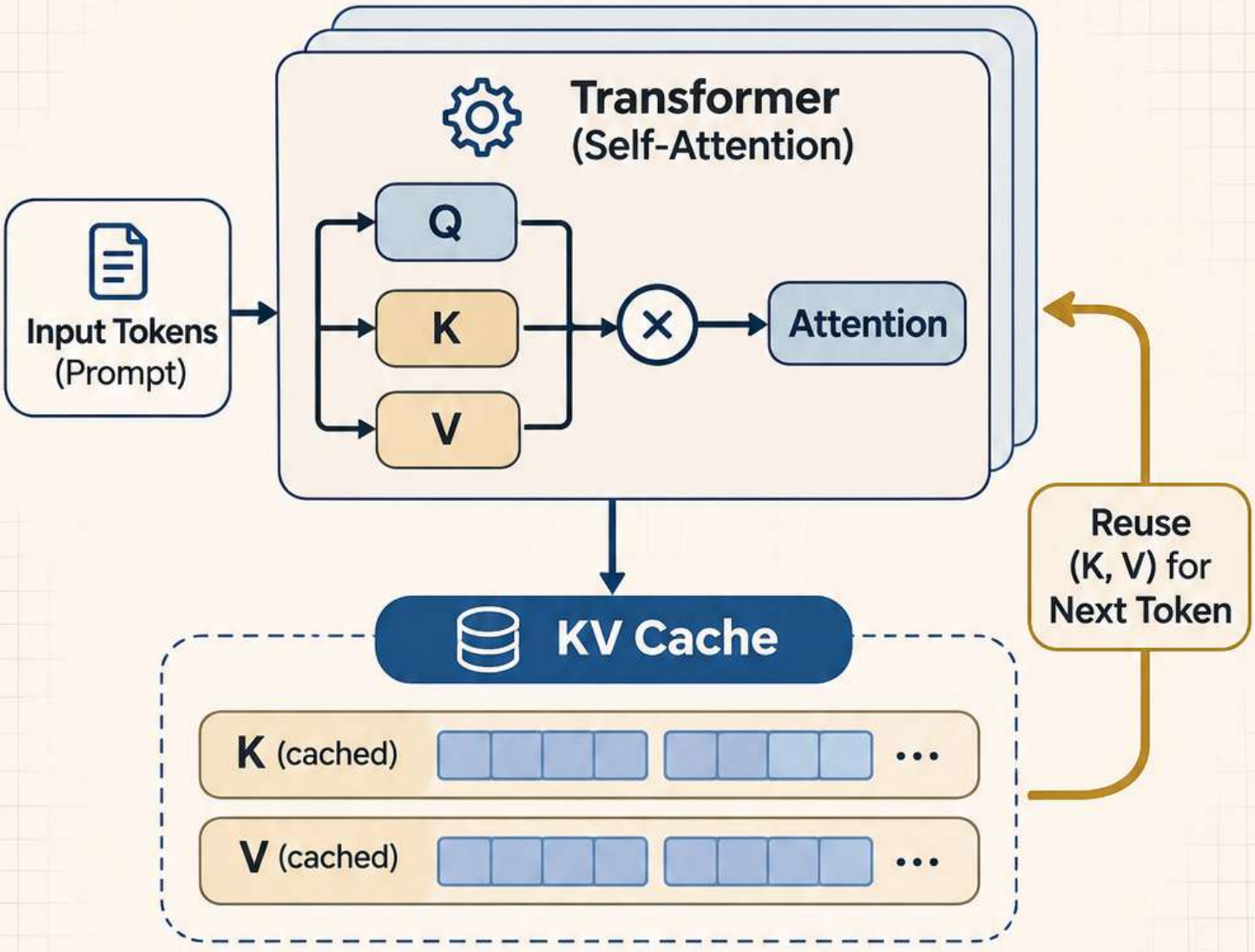
KV Cache & LLM Inference



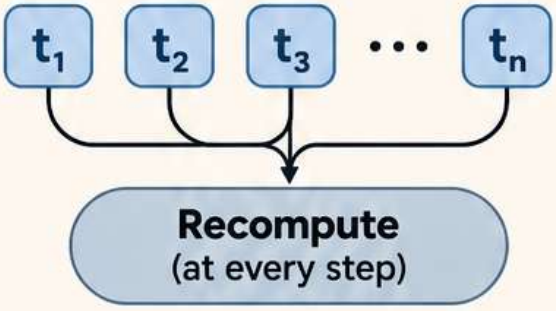
Faster Inference | **Lower Compute** | **Higher Throughput**

VANSH BUDHIRAJA

Why LLMs Need KV Cache?

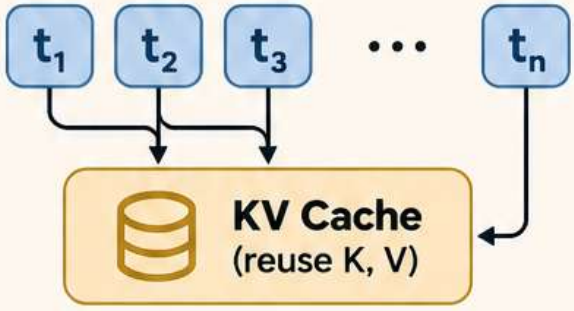
Without KV Cache

For each new token, the model recomputes attention for all previous tokens again.



With KV Cache

Store the Key (K) and Value (V) from previous tokens and reuse them for new tokens.



The Inference Process

Prefill
(Process the prompt)

→

Decode
(Generate tokens one by one)

Key Benefits

Faster Inference
Avoids redundant computation.

Lower Compute Cost
Reuses previously computed states.

Higher Throughput
Generates more tokens per second.

Key Takeaway

KV Cache turns inference from recomputing everything to reusing past attention states, making generation much faster and more efficient.

Understanding Key and Value Caching

Q, K, V in Self-Attention

Q — **Query**
What the current token is looking for.

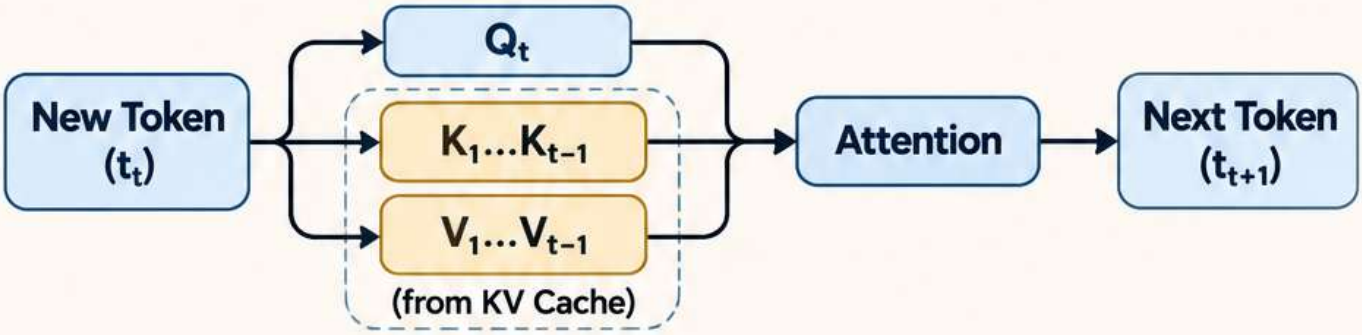
K — **Key**
A representation of previous tokens.

V — **Value**
The information (content) from previous tokens.

What is KV Caching?

It stores the Key (K) and Value (V) states computed for previous tokens and **reuses them** during generation.

How It Works



Key Takeaways

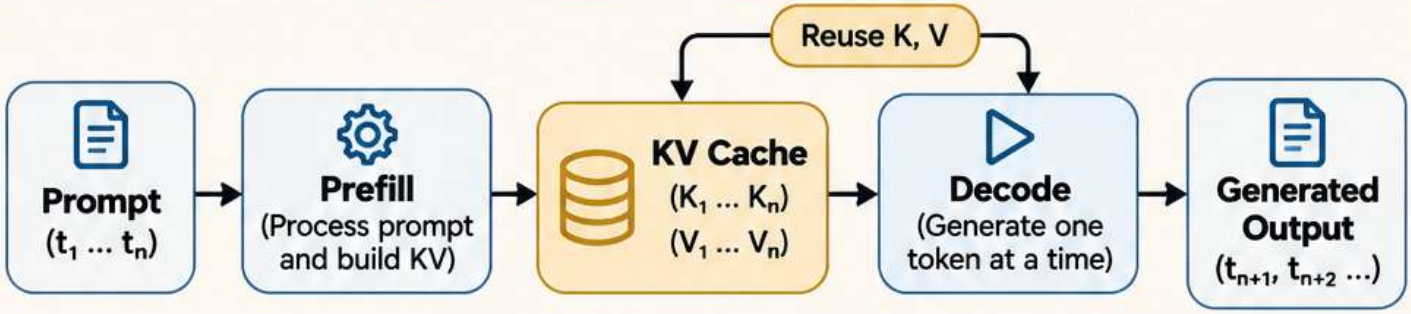
- The KV Cache stores K and V for all processed tokens.
- During decoding, only a new Q is computed.
- Past K and V are reused to compute attention efficiently.

Why It Helps

- **Avoids recomputing** previous states.
- **Reduces compute** per generated token.
- **Enables faster and** more scalable inference.

How KV Cache Works During Inference

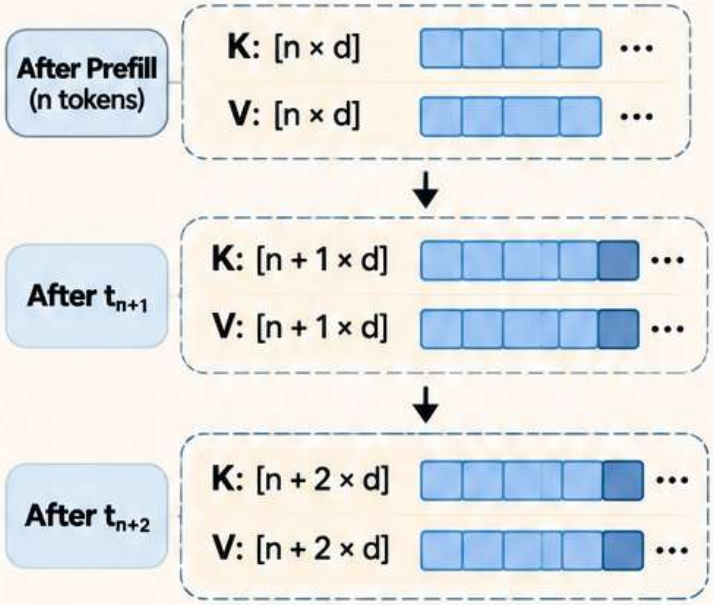
Inference Flow (End-to-End)



Step-by-Step Process

1. **Process Prompt**
   Run the full prompt through the model (prefill).
2. **Create KV Cache**
   Compute and store K and V for all prompt tokens.
3. **Decode Token**
   Generate the next token using the cached K and V.
4. **Reuse & Repeat**
   Keep the existing K and V, compute only the new token's K and V, and append to cache.

KV Cache Growth During Decoding



Key Takeaways

One Full Pass for the Prompt
Compute K and V once during prefill.

Incremental Generation
Only compute K and V for the new token.

Efficient Reuse
Reuse cached states to make decoding much faster.

KV Cache Memory Growth

Why Does It Grow?

The KV cache stores K and V for every token, at every layer, for all attention heads.

As the sequence length increases, the cache size grows linearly.

KV Cache Memory Formula

$$\text{Memory} = 2 \times L \times H \times S \times D_h \times B \times P$$

- **L** = Number of layers
- **H** = Number of heads
- **S** = Sequence length (tokens)
- **D_h** = Head dimension (= hidden_size / H)
- **B** = Batch size
- **P** = Bytes per value (2 for FP16, 1 for INT8)

How Memory Increases



What Affects Cache Size?

Layers (L)
More layers → larger cache.

Attention Heads (H)
More heads → more K and V to store.

Sequence Length (S)
Longer context → linear growth.

Batch Size (B) & Precision (P)
Larger batches or higher precision (FP16/FP32) → higher memory use.

Example Calculation

L = 32, H = 32, S = 8,192
d_h = 128, B = 1, P = 2 (FP16)

Memory
≈ 4.0 GB

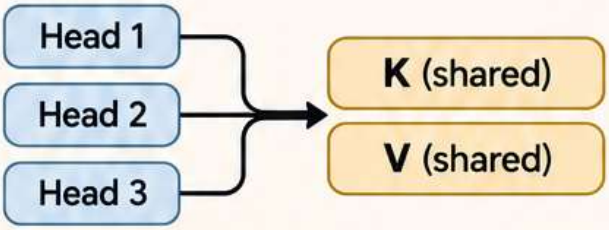
Key Takeaway

KV cache improves speed, but it can consume significant GPU memory, especially with longer context, larger models, or higher batch sizes.

KV Cache Optimization Techniques

Multi-Query Attention (MQA)

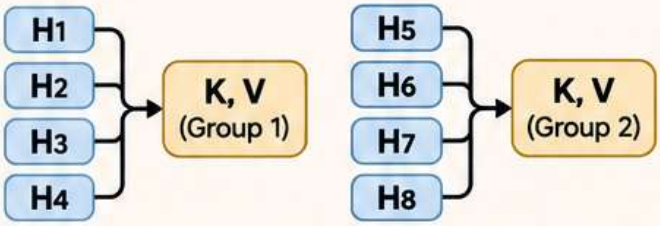
Use **one shared K and V across all attention heads** (instead of separate K, V per head).



Benefit: Up to **H×** lower KV memory (H = number of heads).

Grouped-Query Attention (GQA)

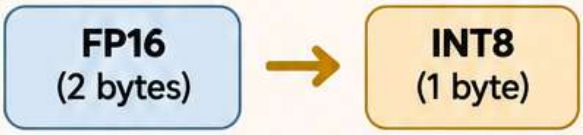
A middle ground — **group multiple heads to share the same K and V** (e.g., 4 heads per group).



Benefit: Lower memory than standard attention, with better quality than MQA.

Quantized KV Cache

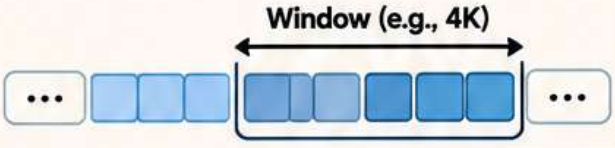
Store K and V in lower precision (e.g., FP8, INT8) instead of FP16/FP32.



Benefit: **~2× smaller cache** with minimal impact on output quality.

Sliding Window Attention

Keep KV cache only for a recent window of tokens (e.g., last 4K tokens).



Benefit: Bounded and predictable memory, ideal for long-context inference.

Overall Impact

Lower Memory Use

Significantly reduces KV cache size.

Higher Throughput

Fits larger context or more requests on the same GPU.

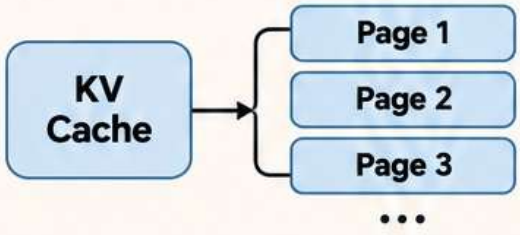
Scalable Inference

Enables efficient and cost-effective deployment at scale.

Advanced KV Cache Management

PagedAttention

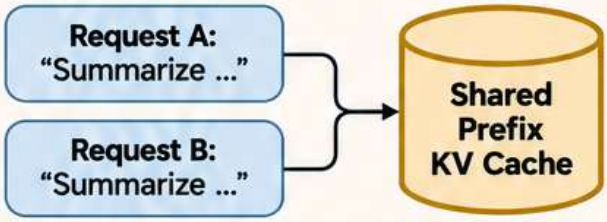
Stores KV cache in fixed-size pages (blocks) instead of one large contiguous memory.



Benefit: Better memory utilization, less fragmentation, and higher serving throughput (e.g., vLLM, TensorRT-LLM).

Prefix Caching

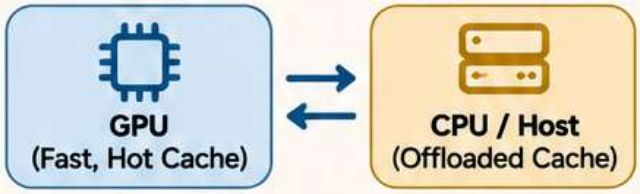
Reuses KV cache for a common prompt prefix across multiple requests.



Benefit: Avoids recomputing the same prefix, reducing latency and compute cost (e.g., multi-user and RAG setups).

Cache Sharing & Offloading

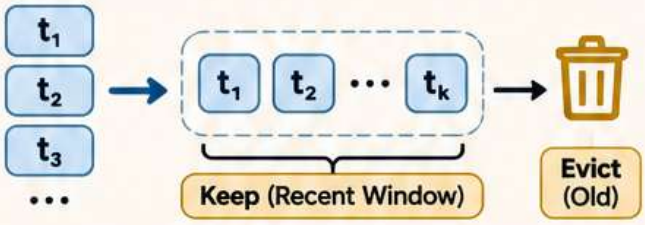
Share or move KV cache across devices/tiers to manage limited GPU memory.



Benefit: Handles larger contexts and higher concurrency by moving rarely used cache to CPU or host memory.

Eviction & Memory Management

Intelligently remove or compress less useful KV cache to stay within the memory budget.



Benefit: Keeps memory bounded while maintaining output quality (e.g., LRU, sliding window, or importance-based eviction).

Key Takeaways

Use the Right Strategy

Combine techniques based on your model size, context length, and workload.

Balance Speed and Memory

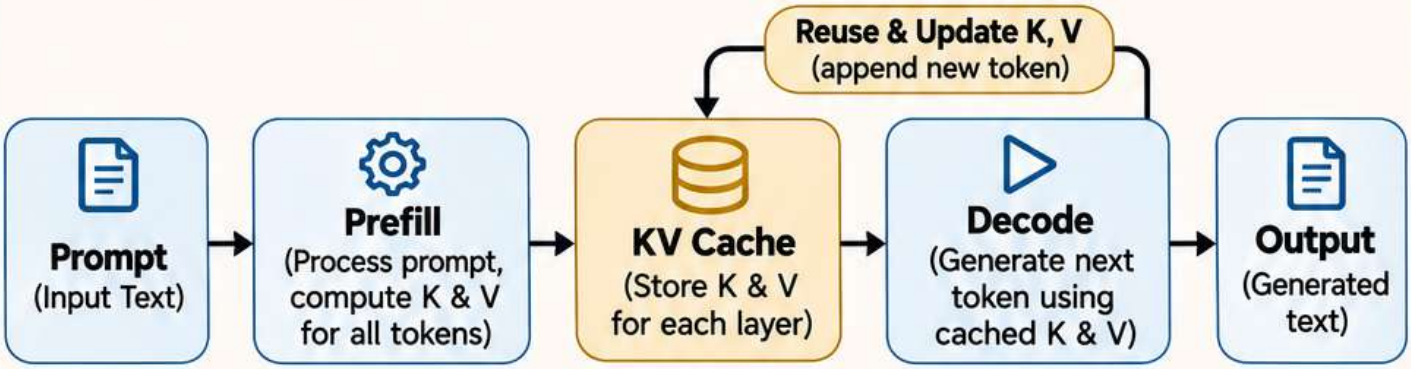
Maximize throughput without exceeding GPU limits.

Production Ready

These techniques make LLM inference more efficient, scalable, and cost-effective.

Complete KV Cache Inference Architecture

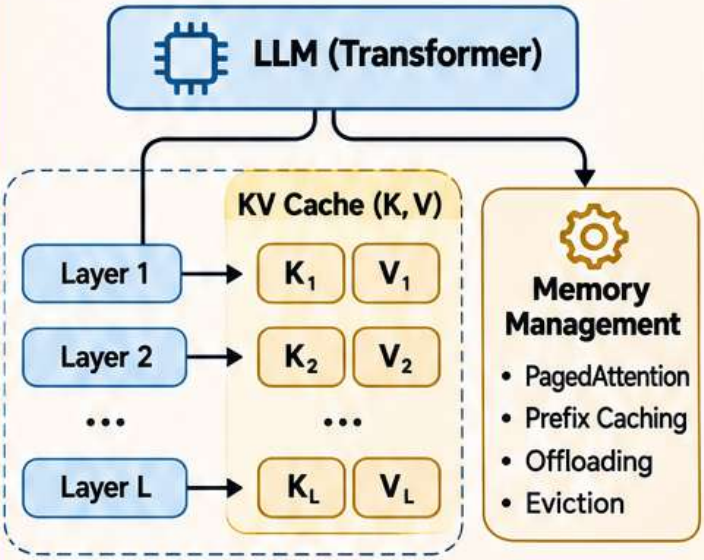
End-to-End Inference Flow



Detailed Workflow

1. **Prefill (One-time)**
   Run the entire prompt through the model and create initial K & V.
2. **Cache Reuse**
   During decoding, reuse the stored K & V for past tokens.
3. **Incremental Update**
   Compute K & V only for the new token and append to cache.
4. **Repeat**
   Continue until the stop condition (e.g., max tokens or EOS) is met.

Overall Architecture



Key Takeaways

Faster Inference
Avoids recomputing previous attention states.

Lower Compute Cost
Computes K & V only for new tokens.

Memory Efficient
Use techniques like GQA, quantization, and paging.

Production Ready
Enables scalable, low-latency LLM deployment.